Catalytic search for a common accepting input of dense read-once branching programs

Edward Pyne

doi:10.4230/LIPIcs.CCC.2024.4; CC BY 4.0

Editorial scope. The counted result is original Theorem 7. Original result numbers are explicit. Other included statements and proofs are uncounted source-local support.

Allowed standard prerequisites (editorial).

- Nisan pairwise-independent hash mixing and hybrid-generator bound; canonicalizer specialization of Putterman-Pyne. Original Lemmas 22,29,31 and bibliography [16],[19]; author local proofs/translations of Lemmas 15,21 and Proposition25 included.

Definition 1. Let $\mathsf{CTISP}[T(n), S(n), C(n)]$ be the set of languages recognized by catalytic machines that use $O(S(n))$ workspace and $O(C(n))$ catalytic space on inputs of size n, and run in time $\mathrm{poly}(T(n))$ in the worst case. Note that the worst-case runtime must hold over every catalytic tape, as well as every input.

Definition 4. A read-once branching program (ROBP) B of width w and length n and alphabet $\{0,1\}^t$ is defined by a function $B : [w] \times \{0,1\}^t \to [w]$.[1] For $x \in (\{0,1\}^t)^n$, define

$$B[i, x] = B[B[\ldots B[B[i, x_1], x_2] \ldots], x_{n-1}], x_n].$$

It is well known that to derandomize BPL, it suffices to estimate $\Pr_{x \leftarrow U_n}[B[1, x] = 1]$ up to error $1/3$ for an ROBP B of length n, width n and alphabet $\{0,1\}$.

Definition 6 (Mutual ROBP Hitting). Given a list of branching programs $(B^1, \ldots, B^n)$ each of length and width n, such that $\mathbb{E}[B^j(U_n)] \geq 1 - 1/n^2$ for every j, produce a fixed $x \in \{0,1\}^n$ such that

$$\bigwedge_{j \in [n]} B^j(x) = 1.$$

Theorem 7. There is a $\mathsf{CTISP}[n, \log n, \log^2 n]$ algorithm that solves the mutual ROBP hitting problem.

[1] The standard definition of ROBPs permits the transition function to differ between the layers. However, as we will always be dealing with programs where $w \geq n$, and we are insensitive to polynomial losses in the width, we can assume all transition functions are the same for clarity.

Definition 9 (Catalytic Turing Machine [3]). A Turing machine M is a catalytic machine using time $T(n)$, workspace $S(n)$, and catalytic space $C(n)$ if it has a work tape, a read-only input tape, a write-only output tape, and a catalytic tape $\mathbf{w}$. We require that for every input x with $|x| = n$ and every $\mathbf{w}$, $M^{\mathbf{w}}(x)$ halts in time at most $T(n)$, using at most $S(n)$ cells on the worktape and $C(n)$ cells on $\mathbf{w}$. Moreover, the final configuration of $\mathbf{w}$ must be equal to its initial configuration, for every x and $\mathbf{w}$. We now define the notion of a catalytic machine that computes a function. We furthermore define the notion of partially computing a function, where on some tapes $\mathbf{w}$ the machine can output a special failure symbol $\perp$.

Definition 10. For a function $f : \{0,1\}^* \to \{0,1\}^*$, we say a catalytic machine M (catalytically) computes f if for every x and $\mathbf{w}$, $M^{\mathbf{w}}(x) = f(x)$, and at the end of the computation $\mathbf{w}$ is in its original state. We say that M partially (catalytically) computes f if for every x and $\mathbf{w}$, $M^{\mathbf{w}}(x) \in \{\perp, f(x)\}$, and at the end of the computation (no matter the output) $\mathbf{w}$ is in its original state. Partial catalytic computation is trivial without further restrictions (as M can always output $\perp$), but we require it as an intermediate step in our analyses. We require a further condition on our machines, that they can revert the catalytic tape at any time without the catalytic tape traversing any new configurations:[2]

Definition 11. A catalytic machine M is reversible if for every x and initial configuration $\mathbf{w}$, at any point during the execution of $M^{\mathbf{w}}(x)$, the machine can receive an external REVERT signal. Let $P = P(\mathbf{w})$ denote all prior configurations of the catalytic tape during the execution of $M^{\mathbf{w}}(x)$. After this signal, M must reset $\mathbf{w}$ to the original configuration, and moreover every intermediate configuration of $\mathbf{w}$ during this process must lie in P. We require any time bound on M to hold even in the case that M is given the REVERT command at an arbitrary point.

Catalytic Derandomization: author Section 3

In this section we prove Theorem 2. We state all our results in terms of catalytic algorithms for the stochastic matrix powering problem, as it is easily compatible with the recursive framework we implement later. Recall a nonnegative matrix is stochastic (resp. substochastic) if all row sums are 1 (resp. at most 1). For a set S, let U_S be the uniform distribution over S, and let $U_n = U_{\{0,1\}^n}$.

Theorem 14. There is a $\mathsf{CTISP}[n, \log n, \log^2 n]$ algorithm that, given n and a stochastic matrix $M \in [0,1]^{n \times n}$ where each entry is specified with $O(\log n)$ bits of precision, outputs $\widetilde{M} \in [0,1]^{n \times n}$ such that $\|\widetilde{M} - M^n\|_1 \leq 1/n$. We recall the existence of efficient algorithms which canonicalize (sub)stochastic matrices, essentially reducing the stochastic matrix powering problem to producing a PRG that fools a branching program.

[2] There are existing results related to transforming catalytic algorithms into reversible catalytic algorithms [15]. However, they do not appear to maintain worst-case runtime over the catalytic tape, which is crucial for our results.

Lemma 15 ([21,19,7]). There is a constant $c > 0$ and a space $O(\log(nw/\varepsilon))$ algorithm which, given $\varepsilon > 0$ and $n, w \in \mathbb{N}$ where $w \geq n$ and a substochastic matrix $M \in [0,1]^{w \times w}$ with $O(\log w)$ bits of precision, returns a branching program

$$B : [(w/\varepsilon)^c] \times \{0,1\}^m \to [(w/\varepsilon)^c]$$

of width $w' = (w/\varepsilon)^c$ and length $m = n \cdot O(\log(w/\varepsilon))$ where m is a power of two. Moreover, letting $\widetilde{M} \in [0,1]^{w \times w}$ be the (substochastic) matrix where for $i, j \in [w]$ we define[3] $\widetilde{M}_{i,j} = \Pr_{x \leftarrow U_n}[B[i,x] = j]$, we have $\|\widetilde{M} - M^n\|_1 \leq \varepsilon$. As this is not the way these results are stated, we provide a translation in Appendix A. We next define the Nisan PRG, and recall several auxiliary lemmas.

The Nisan PRG

Given a branching program, we first define the larger alphabet program obtained from duplicating each edge:

Definition 16. For $t \in \mathbb{N}$, for $B : [w] \times \{0,1\}^n \to [w]$ of width w, let $B^t : [w] \times (\{0,1\}^t)^n \to [w]$ be the branching program of length n and width w over alphabet $\{0,1\}^t$ with transition function $B_t[a, y] = B[a, y_1]$, where y_1 is the first bit of $y \in \{0,1\}^t$. Note that B^t can be constructed in space $O(\log(tnw))$ given B, and furthermore for every $i, j \in [w]$, $\Pr_{x \leftarrow U_n}[B[i,x] = j] = \Pr_{x \leftarrow U_{(\{0,1\}^t)^n}}[B^t[i,x] = j]$. We recall a pairwise independent hash family with a very efficient description:

Observation 17. For every $t \in \mathbb{N}$, there exists a pairwise independent hash family $H : \{0,1\}^t \to \{0,1\}^t$ such that $|H| = 2^{2t}$, and $h \in H$ (which we associate with $h \in \{0,1\}^{2t}$) can be evaluated in space $O(t)$. Given a (hash) function $h : \{0,1\}^t \to \{0,1\}^t$ and a program B, we define an operator that applies a single level of the Nisan construction with hash function h.

Definition 18. Given $B^t : [w] \times (\{0,1\}^t)^n \to [w]$ and $h : \{0,1\}^t \to \{0,1\}^t$, let $B^{t,h} : [w] \times (\{0,1\}^t)^{n/2} \to [w]$ be the width w, length $n/2$ program with transition function

$$B_{t,h}[a, x] = B_t[B_t[a, x], h(x)].$$

Using a recursive application of hash functions, we can define the Nisan PRG as follows.

Definition 19. For $(h_1, \ldots, h_\ell) \in H_t$, define $\mathrm{NIS}_{(h_1,\ldots,h_\ell)} : \{0,1\}^t \to \{0,1\}^{t \cdot n}$ inductively as follows. Let $\mathrm{NIS}_0(x) = x_1$, and for $j \in [\ell]$

$$\mathrm{NIS}_{(h_1,\ldots,h_j)}(x) = (\mathrm{NIS}_{(h_1,\ldots,h_{j-1})}(x) \| \, \mathrm{NIS}_{(h_1,\ldots,h_{j-1})}(h_j(x))).$$

Note that $B[\cdot, \mathrm{NIS}_{(h_1,\ldots,h_\ell)}(\cdot)]$ and $B_{t,(h_1,\ldots,h_\ell)}[\cdot,\cdot]$ (as defined in Definition 18) are equal as functions. To analyze the Nisan PRG, we define the notion of a hash function being good for composing two functions, and a PRG being good for a function.

[3] Note that this truncates the size from w' back to w.

Definition 20. For every $n, w, t \in \mathbb{N}$ and $\delta > 0$ and $f : [w] \times (\{0,1\}^t)^n \to [w]$ and $G : \{0,1\}^t \to (\{0,1\}^t)^n$, we say that G is δ-good for f if for every $i, j \in [w]$,

$$\left| \Pr_{x \leftarrow U_t}[f[i, G(x)] = j] - \Pr_{x \leftarrow U_{(\{0,1\}^t)^n}}[f[i, x] = j] \right| \leq \delta.$$

Moreover, we say $h \in H$ is δ-good (and δ-bad otherwise) for $f : [w] \times \{0,1\}^t \to [w]$ if $G(x) = (x \| h(x))$ is δ-good for $f[i, (x, y)] = f[f[i, x], y]$. We recall that a random hash function is good with high probability.

Lemma 21 ([16]). For every f, $\Pr_{h \leftarrow H_t}[h \text{ is } \delta\text{-good for } f] \geq 1 - w^5(1/\delta)^2/2^t$. (We provide a proof in Appendix A.) Moreover, a hybrid argument establishes the following.

Lemma 22 ([16]). For every $B^t : [w] \times (\{0,1\}^t)^n \to [w]$ and $\vec{h} = (h_1, \ldots, h_\ell)$, suppose for every $i \in [\ell]$, h_i is δ-good for $B_{t,h_1,\ldots,h_{i-1}}$. Then $\mathrm{NIS}_{\vec{h}}$ is $\delta \cdot nw$-good for B.

Catalytic Derandomization

We now state the main result that powers both of our derandomizations.

Theorem 23. There is a pair of reversible catalytic algorithms A, D that run in workspace $O(\log(nw/\varepsilon))$, catalytic space $O(\log(n) \cdot \log(nw/\varepsilon))$, and time $\mathrm{poly}(nw/\varepsilon)$ and act as follows. Given $\varepsilon > 0$ and a length $n = 2^\ell$, width w ROBP $B : [w] \times \{0,1\}^n \to [w]$ where $w \geq n$:

- The machine $A^{\mathbf{w}}(B)$ outputs $V \in \{0,1\}^{2\ell}$ and $t = O(\log(nw/\varepsilon))$ and sets the catalytic tape to $\mathbf{w}'$, such that $(\mathbf{w}', V)$ contains a (read-only) data structure supporting access to hash functions $\vec{h} = (h_1, \ldots, h_\ell)$ each on t bits, such that $\mathrm{NIS}_{\vec{h}}$ is ε-good for B.
- The machine $D^{\mathbf{w}'}(B, V)$ sets the final catalytic tape configuration to $\mathbf{w}$.

For the extension to search problems, we require a version which takes in a list of ROBP, and constructs a set of hash functions that is simultaneously good for all of them. Our proof is for this more general notion, which immediately implies Theorem 23:

Theorem 24. There is a pair of reversible catalytic algorithms A, D that run in workspace $O(\log(nw/\varepsilon))$, catalytic space $O(\log(n) \cdot \log(nw/\varepsilon))$, and time $\mathrm{poly}(nw/\varepsilon)$ and act as follows. Given $\varepsilon > 0$ and a set of w length $n = 2^\ell$, width w ROBPs

$$\mathbf{B} = (B^1, \ldots, B^w)$$

with $B^j : [w] \times \{0,1\}^n \to [w]$ where $w \geq n$:

- The machine $A^{\mathbf{w}}(\mathbf{B})$ outputs $V \in \{0,1\}^{2\ell}$ and $t = O(\log(nw/\varepsilon))$ and sets the catalytic tape to $\mathbf{w}'$, such that $(\mathbf{w}', V)$ contains a (read-only) data structure supporting access to hash functions $\vec{h} = (h_1, \ldots, h_\ell)$ each on t bits, such that $\mathrm{NIS}_{\vec{h}}$ is ε-good for B^j for every $j \in [w]$.

- The machine $D^{\mathbf{w}'}(\mathbf{B}, V)$ sets the final catalytic tape configuration to $\mathbf{w}$.

To make our compression and decompression algorithms work, we require that we can determine if a hash function is good for a branching program at a certain level of the Nisan construction, given pointers to the hash functions at the previous levels:

Proposition 25. There is a space $O(t + \log(w/\delta))$ algorithm that, given $n, w, t \in \mathbb{N}$ with $w \geq n$ and $\widetilde{h} \in \{0,1\}^{2t}$ and $B : [w] \times \{0,1\}^n \to [w]$ and (read only) $\mathbf{w}$ and pointers $p_1, \ldots, p_r$ such that $\mathbf{w}[p_i, \ldots, p_i + 2t] = h_i$ represents a hash function on t bits, returns if $\widetilde{h}$ is δ-good for $B_{t,(h_1,\ldots,h_r)}$. We give a proof in Appendix A, as it essentially follows from the argument of [18]. We can then prove the theorem:

Proof of Theorem 24. We assume without loss of generality that n, w and $1/\varepsilon$ are powers of two. Set

$$t_0 = 60\log(w/\varepsilon), \qquad t_1 = 25\log(w/\varepsilon), \qquad \delta = \varepsilon/nw \geq \varepsilon/w^2,$$

and note that we choose t_1 large enough such that a good series of hash functions (for all B^j simultaneously) on t_1 bits always exists. The algorithm works as follows. First, virtually divide the catalytic tape as:

$$\mathbf{w} = (\mathbf{w}_1 \| \mathbf{w}_2 \| \cdots \| \mathbf{w}_{2\ell})$$

where $|\mathbf{w}_i| = 2t_0$, which we think of as initially holding $h : \{0,1\}^{t_0} \to \{0,1\}^{t_0}$. Note that $|\mathbf{w}| = \ell \cdot 4t_0 = O(\log(n)\log(nw/\varepsilon))$ as claimed. Next, initialize $V \in \{0,1,*\}^{2\ell}$ to indicate if each block is compressed, uncompressed, or unprocessed respectively. The first two cases correspond to the following two formats of the block:

$$\mathbf{w}_i = \begin{cases} h & V_i = 1 \\ (z \| j \| 0^{50\log(w/\varepsilon)}) & V_i = 0. \end{cases}$$

Informally, the first corresponds to block i originally containing a good hash function for $\mathbf{B}$, and the second corresponds to block i originally containing a bad hash function for B^j, which is thus compressible (in fact, z represents a compressed version of the original data). We define notation for the set of blocks in each configuration:

Definition 26. For $b \in \{0,1\}$, let $I^b(V) \subseteq [2\ell]$ correspond to the indices such that $V_i = b$, and let $S^b(V) = |I^b(V)|$. Next, we initialize a counter $i = 1$ for the current block. We then iterate over $i = \{1, \ldots, 2\ell\}$ until $\max\{S^1(V), S^0(V)\} = \ell$.[4] For each i, the algorithm works as follows. Let $\widetilde{h} = \mathbf{w}_i$ be the hash function (on t_0 bits) obtained from the current block. We then test if $\widetilde{h}$ is δ-good for

$$f^j = B^j_{t_0,\vec{h}_p}, \qquad \text{for every } j \in [w].$$

Where

$$\vec{h}_p = (\mathbf{w}_{I^1(V)_1}, \ldots, \mathbf{w}_{I^1(V)_{S^1(V)}})$$

[4] If we exit before $i = 2\ell$, set the remaining indices of V to an arbitrary value, which we ignore for clarity of presentation.

corresponds to the hash functions on the preceding good blocks, and $B^j_{t_0,\vec{h}_p}$ is defined as in Definition 18 applied to B^j and $\vec{h}_p$. As the index set $I^1(V)$ is easy to generate given V, this test can be performed in space $O(\log(nw/\varepsilon))$ without modifying the catalytic tape (and hence also in time $\text{poly}(nw/\varepsilon)$), by Proposition 25. Given the results of this test, we break into cases depending on if $\widetilde{h}$ is good:

- If $\widetilde{h}$ is δ-good for f^j for every $j \in [w]$, set $V_i = 1$.
- If $\widetilde{h}$ is δ-bad for some f^j, set $V_i = 0$ and let j be the first index for which this holds. Next, by enumeration over strings $h \in \{0,1\}^{2t_0}$ (which we can do using the workspace), determine the index of $\widetilde{h}$ in the set

  $$\text{BAD}_{i,j} = \{h \in \{0,1\}^{2t} : h \text{ is } \delta\text{-bad for } B^j_{t_0,\vec{h}_p}\}$$

  where we again perform this test using Proposition 25. Letting the index of $\widetilde{h}$ in this set be z, write

  $$\mathbf{w}_i = (z \| j \| 0^{50\log(w/\varepsilon)})$$

  (we perform this write operation left to right, and will revert it right to left). We denote the final $50\log(w/\varepsilon)$ bits as free space. Finally, we claim that we can in fact write these quantities in space $|\mathbf{w}_i| = 2t_0$. The index j requires $\log(w)$ bits. Moreover, we have

  $$\begin{aligned} |\,\text{BAD}_{i,j}\,| &= 2^{2t_0} \cdot \Pr_{h \leftarrow H}[h \text{ is } \delta\text{-bad for } f] \\ &\le 2^{2t_0} \cdot w^5(1/\delta)^2/2^{t_0} \qquad \text{(Lemma 21)} \\ &\le 2^{2t_0} \cdot (w/\varepsilon)^{7-60}. \end{aligned}$$

  And thus $\log|\,\text{BAD}_{i,j}\,| \le 2t_0 - 51\log(w/\varepsilon) = |\mathbf{w}_i| - 51\log(w/\varepsilon)$. Therefore, we can record all required information as claimed.

After processing all blocks, we obtain a catalytic tape $\mathbf{w}'$ and one of two cases:

- If $S^1(V) = \ell$, there exist $\vec{h} = (h_1, \ldots, h_\ell)$ corresponding to the hash functions (on t_0 bits) in $I^1(V)$, and these functions are easy to recover from V.
- Else, we must have $S^0(V) = \ell$. For $i \in I^0(V)$, let F_i be the $50\log(w/\varepsilon)$ free bits in $\mathbf{w}_i$. Note that a description of a hash function $h : \{0,1\}^{t_1} \to \{0,1\}^{t_1}$ is of size $|F_i|$. Iterating over $i \in I^0(V)$ in increasing order, we find (via brute force enumeration) a hash function $\widetilde{h}$ that is δ-good for

  $$f^j = B^j_{t_1,\vec{h}_p}, \quad \text{for every } j \in [w].$$

  Where

  $$\vec{h}_p = (\mathbf{w}_{F_{I^0(V)_1}}, \ldots, \mathbf{w}_{F_{I^0(V)_{i-1}}})$$

  corresponds to the (δ-good) hash functions stored on the free space in the preceding indices of $I^0(V)$. Next, store $\widetilde{h}$ in $\mathbf{w}_{F_i}$. Such a good hash function always exists, by our choice of t_1 and Lemma 21, and moreover

testing if each candidate is good can be computed in the desired space and time by Proposition 25. After this processing,

$$(\mathbf{w}_{F_{I^0(V)_1}}, \ldots, \mathbf{w}_{F_{I^0(V)_\ell}})$$

contain ℓ good hash functions, which we can clearly access in read-only fashion given given V and $\mathbf{w}'$.

Thus, in both cases we obtain a set of hash functions $\vec{h} = (h_1, \ldots, h_\ell)$ on $t = O(\log(nw/\varepsilon))$ bits that is δ-good for every one of the relevant tests, so by Lemma 22 we have that $\mathrm{NIS}_{\vec{h}}$ is $\delta \cdot nw \leq \varepsilon$-good for B^j for every j.

Decompression and Reversibility. It suffices to show that at any point, the algorithm can revert the tape to the original configuration $\mathbf{w}$ (and then $D(\mathbf{B}, V)$ simply issues the REVERT command). No matter the present configuration, we iterate through $I^0(V)$ in descending order. Letting the current index be $i \in I^0(V)$, recall this block is of form $(\mathbf{w}')_i = (z \| j \| *)$. First write $0^{50\log(w/\varepsilon)}$ to the last indices (in reverse order to satisfy reversibility), such that we reach the configuration after compressing the block. Then enumerate over $h \in \{0,1\}^{2t_0}$ using workspace $O(t_0 + \log(w/\varepsilon))$, until we find the hash with index z in $\mathrm{BAD}_{i,j}$, where $\mathrm{BAD}_{i,j}$ and $B^j_{t_0,\vec{h}_p}$ are defined as before (which we still have access to because we reset the tape in reverse order), and we determine membership by Proposition 25. Once we find this h, write $(\mathbf{w}')_i = h$ (in the reverse order to satisfy reversibility) and proceed to the next highest index in $I^0(V)$. After this process has completed, it is clear from construction that $\mathbf{w}$ has been reset to the original configuration, and that the tape never reaches a new intermediate configuration during this process.

Time and Space. In every step of the computation, we perform at most $\mathrm{poly}(2^{t_0} nw/\varepsilon)$ work to determine if a hash function is good, find the index of a bad hash function, or find a good hash function. Moreover, as at every point we store at most a constant number of hash functions on the worktape, the space consumption follows. ◁ It is easy to go from Theorem 23 to Theorem 14.

Proof of Theorem 14. Let $B : [\mathrm{poly}(n)] \times \{0,1\}^{n^c} \to [\mathrm{poly}(n)]$ be the ROBP obtained from applying Lemma 15 to M with $n = n$, $w = n$, and $\varepsilon = 1/2n$. We then call Theorem 23 with $B = B$ and $\varepsilon = 1/2n^2$. Let $\vec{h} = (h_1, \ldots, h_{c\log n})$ be the hash functions obtained from this call, which we have implicit access to via the current state of the catalytic tape $\mathbf{w}'$ and V, and let $t = O(\log n)$ be the domain of the hash functions. Then enumerate over $x \in \{0,1\}^t$ and for $i, j \in [w]$ let

$$\widetilde{M}_{i,j} = \Pr_{x \leftarrow U_t}[B[i, \mathrm{NIS}_{\vec{h}}(x)] = j].$$

By Lemma 15 and Theorem 23, we have the guarantee that

$$\|\widetilde{M} - M^n\|_1 \leq \frac{1}{2n} + n \cdot \frac{1}{2n^2} = 1/n.$$

◁ Finally, we can use Theorem 24 to prove Theorem 7.

Proof of Theorem 7. The algorithm works as follows. Given $\mathbf{B} = (B^1, \ldots, B^n)$ of width and length n where $\mathbb{E}[B^j(U_n)] \geq 1 - 1/n^2$ for every j, we call Theorem 24 with $\mathbf{B} = \mathbf{B}$ and $\varepsilon = 1/2n^2$. Let $\vec{h} = (h_1, \ldots, h_{\log n})$ be the hash functions obtained from this call, which we have implicit access to via the current state of the catalytic tape $\mathbf{w}'$ and V, and let $t = O(\log n)$ be the length of the domain of the hash functions. We then claim there is some $x \in \{0,1\}^t$ such that for all $j \in [n]$,

$$B^j(\mathrm{NIS}_{\vec{h}}(x)) = 1.$$

We have this as

$$\begin{aligned}
\Pr_{x \leftarrow U_t}\left[\bigwedge_{j \in [n]} B^j(\mathrm{NIS}_{\vec{h}}(x))\right] &\geq 1 - \sum_{j \in [n]} \Pr_{x \leftarrow U_t}[B^j(\mathrm{NIS}_{\vec{h}}(x)) = 0] \\
&\geq 1 - \sum_{j \in [n]} \left(\Pr_{x \leftarrow U_t}[B^j(U_n) = 0] + \frac{1}{2n^2}\right) \\
&\geq 1 - n(2/n^2) > 0
\end{aligned}$$

where the second inequality follows from our choice of ε. Then it is simple to find such an x by enumeration, whereupon we compute and return $\mathrm{NIS}_{\vec{h}}(x) \in \{0,1\}^n$, which is precisely the solution for the mutual ROBP hitting problem. Finally, we use Theorem 24 to reset the tape. ◁

Author Appendix A: proofs used above

We first prove that a hash function drawn from a pairwise independent hash family is good for a function with high probability. To do this, we recall the hash mixing lemma:

Lemma 29 ([16]). Let $A, A' \subseteq \{0,1\}^t$ be arbitrary subsets of density $\rho = |A|/2^t$ and $\rho' = |A'|/2^t$. Then for every $\delta > 0$,

$$\Pr_{h \leftarrow H}\left[\left|\Pr_{x \leftarrow U_t}[x \in A, h(x) \in A'] - \rho\rho'\right| \geq \delta\right] \leq (1/\delta^2)/2^t.$$

Proof of Lemma 21. For every $i, k \in [w]$, let $A_{i,k} = \{x \in \{0,1\}^t : f[i,x] = k\}$ and let $\rho_{i,k} = |A_{i,k}|/2^t$. Note that for every i, j,

$$\Pr_{x,x' \leftarrow U_t}[f[f[i,x],x'] = j] = \sum_{k \in [w]} \rho_{i,k}\rho_{k,j}.$$

Thus, for every h such that for every i, k, j, $|\Pr_{x \leftarrow U_t}[x \in A_{i,k}, h(x) \in A_{k,j}] - \rho_{i,k}\rho_{k,j}| \leq \delta/w$, we have that h is δ-good for f. By Lemma 29 this event occurs with probability $1 - (w/\delta)^2/2^t$ over $h \leftarrow H$ for each pair of sets, and thus with probability $1 - w^3(w/\delta)^2/2^t = 1 - w^5(1/\delta)^2/2^t$ for every tuple (i,j,k). ◁

We recall there is a logspace algorithm which tests if a hash function is good, given oracle access to the function we wish to fool. We remark that there is work [17,6,20] on testing if an entire PRG is good for a branching program, but we need a much weaker claim.

Lemma 30 ([18]). There is a space $O(t + \log(w/\delta))$ algorithm that, given oracle access to $f : [w] \times \{0,1\}^t \to [w]$ and $h \in H_t$ and $\delta > 0$, tests if h is δ-good for f.

Proof. The algorithm enumerates over $i, j \in [w]$. For every i, j, the algorithm computes $p_{i,j} = \mathbb{E}_{x,x' \leftarrow U_t}[f[f[i,x],x']]$ (i.e. the correct probability) by enumeration over x, x' in space $O(t + \log w)$. Then it computes $\widetilde{p}_{i,j} = \mathbb{E}_{x \leftarrow U_t}[f[f[i,x], h(x)]]$ and rejects if the estimate is greater than δ from the true value. Correctness and total space consumption are immediate. $\triangleleft$ We can then prove that we can test if a hash function is good, given B and pointers to preceding hash functions.

Proposition 25. There is a space $O(t + \log(w/\delta))$ algorithm that, given $n, w, t \in \mathbb{N}$ with $w \geq n$ and $\widetilde{h} \in \{0,1\}^{2t}$ and $B : [w] \times \{0,1\}^n \to [w]$ and (read only) $\mathbf{w}$ and pointers $p_1, \ldots, p_r$ such that $\mathbf{w}[p_i, \ldots, p_i + 2t] = h_i$ represents a hash function on t bits, returns if $\widetilde{h}$ is δ-good for $B_{t,(h_1,\ldots,h_r)}$.

Proof. By Lemma 30, it suffices to show that given $i \in [w]$ and $x \in \{0,1\}^t$, we can compute $B_{t,(h_1,\ldots,h_r)}[i,x]$. To do this, the algorithm maintains $v \in [w]$ as its current position in the branching program (initialized to $v = i$) and $i \in [n]$ to track the current layer. To determine the next position, it suffices to determine the ith block of the output of $\mathrm{NIS}_{(h_1,\ldots,h_r)}(x)$. It is well known that this can be computed in space $O(t + \log nw)$ given read-only access to the set of hash functions (by walking down the binary expansion of i, denoted $\langle i \rangle$, and applying h_j if $\langle i \rangle_j = 1$), which we have via the pointers. $\triangleleft$

Author Appendix A: translation of Lemma 15. Finally, we provide a translation of our quantization statement. We first recall a strict specialization of the statement of [19]:

Lemma 31. There exists a canonicalizer algorithm C_t that, given $n, w \in \mathbb{N}$ with $w \geq n$, takes in $\varepsilon > 0$ and a sub-stochastic matrix $M \in \mathbb{R}^{w \times w}$ with each entry represented by at most $O(\log(w/\varepsilon))$ bits, runs in space $O(\log(w/\varepsilon))$, and returns a branching program B of length n and width $w + 1$ with alphabet $\{0,1\}^t$ for $t = O(\log(w/\varepsilon))$. Moreover, letting $\widetilde{M} \in [0,1]^{w \times w}$ be the matrix where for $i, j \in [w]$ we have

$$\widetilde{M}_{i,j} = \Pr_{x \leftarrow U_{(\{0,1\}^t)^n}}[B[i,x] = j]$$

then

$$\|\widetilde{M} - M^n\|_1 \leq \varepsilon.$$

We reduce the alphabet (and slightly increase the length) as follows. We transform B into a branching program of width $(w+1) \cdot 2^t = \mathrm{poly}(w/\varepsilon)$ and length $n \cdot t = n \cdot O(\log(w/\varepsilon))$, where each set of t layers reads t bits, interprets these bits as $\sigma \in \{0,1\}^t$, and takes the transition labeled with σ in B.

References

[1] Dimitris Achlioptas. Database-friendly random projections: Johnson-lindenstrauss with binary coins. J. Comput. Syst. Sci., 66(4):671–687, 2003. doi:10.1016/S0022-0000(03)00025-4.

[2] Sagar Bisoyi, Krishnamoorthy Dinesh, and Jayalal Sarma. On pure space vs catalytic space. Theor. Comput. Sci., 921:112–126, 2022. doi:10.1016/j.tcs.2022.04.005.

[3] Harry Buhrman, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. Computing with a full memory: catalytic space. In Symposium on Theory of Computing, STOC 2014, pages 857–866. ACM, 2014.

[4] Harry Buhrman, Michal Koucký, Bruno Loff, and Florian Speelman. Catalytic space: Nondeterminism and hierarchy. Theory Comput. Syst., 62(1):116–135, 2018. doi:10.1007/ s00224-017-9784-7.

[5] Jin-yi Cai, Venkatesan T. Chakaravarthy, and Dieter van Melkebeek. Time-space tradeoff in derandomizing probabilistic logspace. Theory Comput. Syst., 39(1):189–208, 2006. doi: 10.1007/S00224-005-1264-9.

[6] Kuan Cheng and William M. Hoza. Hitting sets give two-sided derandomization of small space. Theory of Computing, 18(21):1–32, 2022. doi:10.4086/toc.2022.v018a021.

[7] Gil Cohen, Dean Doron, Ori Sberlo, and Amnon Ta-Shma. Approximating iterated multiplication of stochastic matrices in small space. In Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, pages 35–45. ACM, 2023.

[8] James Cook and Ian Mertz. Catalytic approaches to the tree evaluation problem. In Proccedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, pages 752–760. ACM, 2020.

[9] James Cook and Ian Mertz. Tree evaluation is in space o(log n $\cdot$ log log n). Electron. Colloquium Comput. Complex., TR23-174, 2023. arXiv:TR23-174.

[10] Samir Datta, Chetan Gupta, Rahul Jain, Vimal Raj Sharma, and Raghunath Tewari. Randomized and symmetric catalytic computation. In Computer Science – Theory and Applications – 15th International Computer Science Symposium in Russia, CSR 2020, pages 211–223. Springer, 2020.

[11] Dean Doron, Edward Pyne, and Roei Tell. Opening up the distinguisher: A hardness to randomness approach for BPL = L that uses properties of BPL. Electron. Colloquium Comput. Complex., TR23-208, 2023. arXiv:TR23-208.

[12] Chetan Gupta, Rahul Jain, Vimal Raj Sharma, and Raghunath Tewari. Unambiguous catalytic computation. In 39th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2019, volume 150 of LIPIcs, pages 16:1–16:13, 2019.

[13] Daniel M. Kane, Raghu Meka, and Jelani Nelson. Almost optimal explicit johnson-lindenstrauss families. In Leslie Ann Goldberg, Klaus Jansen, R. Ravi, and José D. P. Rolim, editors, Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques – 14th International Workshop, APPROX 2011, and 15th International Workshop, RANDOM 2011, Princeton, NJ, USA, August 17-19, 2011. Proceedings, volume 6845 of Lecture Notes in Computer Science, pages 628–639. Springer, 2011. doi:10.1007/978-3-642-22935-0_53.

[14] Daniel M. Kane and Jelani Nelson. Sparser johnson-lindenstrauss transforms. J. ACM, 61(1):4:1–4:23, 2014. doi:10.1145/2559902.

[15] Ian Mertz. Reusing space: Techniques and open problems. Bulletin of EATCS, 141(3), 2023.

[16] Noam Nisan. Psuedorandom generators for space-bounded computation. In Harriet Ortiz, editor, Proceedings of the 22nd Annual ACM Symposium on Theory of Computing, May 13-17, 1990, Baltimore, Maryland, USA, pages 204–212. ACM, 1990. doi:10.1145/100216.100242.

[17] Noam Nisan. On read-once vs. multiple access to randomness in logspace. Theor. Comput. Sci., 107(1):135–144, 1993. doi:10.1016/0304-3975(93)90258-U.

[18] Noam Nisan. RL $\leq$ SC. Comput. Complex., 4:1–11, 1994. doi:10.1007/BF01205052.

[19] Aaron (Louie) Putterman and Edward Pyne. Near-optimal derandomization of medium-width branching programs. In Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, pages 23–34, 2023.

[20] Edward Pyne, Ran Raz, and Wei Zhan. Certified hardness vs. randomness for log-space. In 64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023, pages 989–1007. IEEE, 2023. doi:10.1109/FOCS57990.2023.00061.

[21] Michael E. Saks and Shiyu Zhou. $\mathsf{BP}_H\mathsf{Space}(S) \subseteq \mathsf{DSPACE}(S^{3/2})$. J. Comput. Syst. Sci., 58(2):376–403, 1999. doi:10.1006/jcss.1998.1616.

[22] D. Sivakumar. Algorithmic derandomization via complexity theory. In John H. Reif, editor, Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada, pages 619–626. ACM, 2002. doi:10.1145/509907.509996.